

Computer Vision

Nicola Conci

a.y. 2025/2026

University of Trento

Davide Donà

April 2026

Notes Repository

Contents

1	Definition of Computer Vision	1
2	Images and Videos	2
2.1	Processing Chain	2
2.2	Digital Images	2
2.2.1	Sampling	3
2.3	Colors	3
2.3.1	RGB	3
2.3.2	Problems of RGB	4
2.3.3	YCbCr	5
2.3.4	HSV	5
2.3.5	Bayer Pattern	5
2.3.6	Quantization	6
2.4	Videos	6
2.4.1	Important Features	6
2.5	Histogram	7
2.5.1	Information from the histogram	7
2.6	Simple Operations on the Histogram	7
2.6.1	Stretching	8
2.6.2	Equalization	8
2.7	Edge Extraction	8
2.7.1	Gradient based approach	9
2.7.2	Practical Example - Sobel Operators	9
2.7.3	Convolution	10
2.8	Smoothing / Enhancement	10
2.8.1	Low-pass filter	10
2.8.2	Gaussian filter	11
2.8.3	Median filtering	11
3	Morphology Filters	12

4	Models	14
4.1	Pinhole Camera Model	14
4.1.1	Features of the model	15
4.2	Projection	16
4.2.1	Perspective Projection	16
4.2.2	Simplified Perspective Projection	17
4.2.3	Orthographic Projection	18
4.3	Illumination Model	19
4.3.1	Reflection	19
4.3.2	Surface Properties	19
4.4	Lenses	20
4.4.1	Thin Lens Equation	21
4.4.2	Focus	22
4.4.3	Autofocus	23
4.4.4	Resolution, Blur and Resolving Power	23
4.4.5	Typical Issues	25
5	Motion detection	26
5.1	Problems	28
5.1.1	Real vs Apparent movement	28
5.1.2	Occlusion	28
5.1.3	Aperture	29
5.2	Optical Flow	29
5.2.1	Optical Flow Function	29
5.3	Motion Detection in practice	30
5.3.1	Change Detection	30
5.3.2	Background subtraction	31
5.3.3	Adaptive Background Subtraction	32
5.3.4	Gaussian Average	32
5.3.5	Improving the Gaussian Average	33
5.3.6	Mixture of Gaussian	33

6	Motion Tracking	36
6.1	2D Tracking Methodologies	36
6.2	Region-Based Tracking	37
6.2.1	Part-Based Models	37
6.2.2	Implementation and Similarity Metrics	37
6.2.3	Addressing Environmental Noise: Shadows	38
6.2.4	Blob Extraction and Post-Processing	38
6.2.5	Target Association	39
6.2.6	Splitting and Merging	40
6.2.7	Occlusion	40
6.3	Feature Based Tracking	41
6.3.1	Good Features to Track	41
6.3.2	Tracking Features	41
6.4	Lucas-Kanade Optical Flow	42
6.4.1	Pyramidal implementation	42
6.5	Bayesian Tracking	44
6.5.1	How it works	45
6.6	Kalman Filter	46
6.6.1	Kalman Filter in Practice	47
6.6.2	The Recursive Stages	48
6.6.3	Extended Kalman Filter	48
6.7	Particle Filter	49
6.7.1	Theoretical Definition	49
7	Camera Geometry	51
7.1	2D Case	51
7.1.1	Affine Transformation	51
7.1.2	Camera Matrix	53
7.1.3	General Camera Matrix	54
7.2	3D Case	55
7.2.1	Calibration	55

7.2.2	3D representation	56
7.2.3	3D affine transformation	57
7.2.4	General Transformation Matrix	59
7.2.5	Camera Model	59
7.2.6	Extension of the Camera Model	59

1 Definition of Computer Vision

Computer Vision is the science and technology of machines that **see**, where "see" means being able to extract information from an image, necessary to perform a task.

2 Images and Videos

Before introducing the concept of images and videos, we first need to define the concept of **signal**, how to represent it and how to process it.

2.1 Processing Chain

The **processing chain** of an image can be represented by the following steps:

- **Acquisition:** The process of transforming a real-world physical signal into a digital one, by means of a **sensor**. In this process, we essentially transfer a portion of the real 3D world into a 2D image. This process is affected by **noise**, which can be added by both the real world and the sensor itself.

Once the signal is acquired, it is transformed using a standard format to a discrete signal, by applying both a **sampling** and a **quantization** process.

- **Enhancement:** The process of applying a transformation to the signal, by applying a **filter** to it, in order to make the picture more suitable for our goal.
- **High-level processing:** The process of extracting information from the signal. This is what we will see in the next chapters.

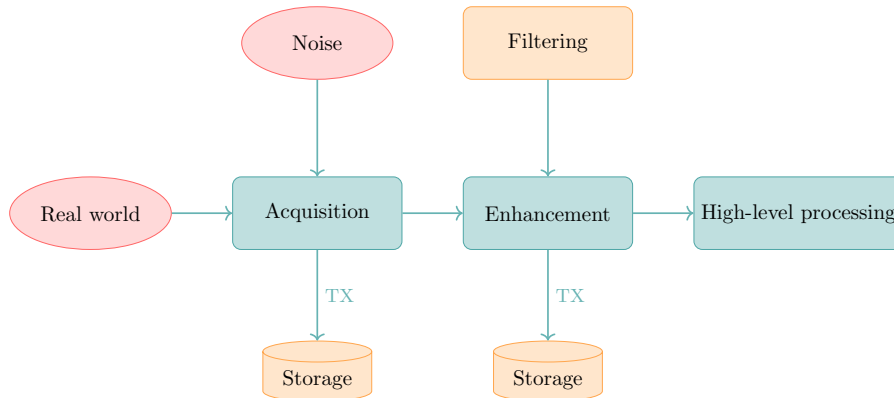


Figure 1: Processing chain of an image.

2.2 Digital Images

A digital image can be seen as a collection of 2D coordinates, called **pixels** (picture elements). Each pixel represents a portion of the real world and can be

gray-scale (black and white, 1 channel) or **color** (typically 3 channels).

2.2.1 Sampling

As we said before, to transform a real-world signal into a digital one, we need to apply a sampling process. When considering video signals, they need to be sampled in all three dimensions: the two spatial dimensions (x and y) and the temporal dimension (t).

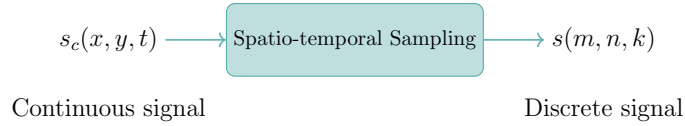


Figure 2: Spatio-temporal sampling of a video signal.

This process is called **spatio-temporal sampling**, and it transforms a continuous signal $s_c(x, y, t)$ into a discrete one $s(m, n, k)$, where m , n and k are the indices of the spatial and temporal dimensions, respectively.

The output of this process is a 2D matrix of pixels. Once we have this, we can manipulate it using many techniques. Pixels are numbered starting from the top-left corner, and the value of a pixel in certain position (c, r) for a given image I is denoted as $I(c, r)$, where c is the column index and r is the row index. The value of a pixel is typically normalized in the range $[0, 1]$.

2.3 Colors

Color is an attribute that humans associate with objects in the real world. Mathematically, we can define it as a combination of different **wavelengths**. Different representations are used to represent colors; we will discuss the most common ones in the following chapters.

2.3.1 RGB

One of the most common representations of colors is the **RGB** (Red, Green, Blue) one. In the *addictive color model*, starting from a black image, we can add different amounts of red, green and blue to it, creating a wide range of colors.

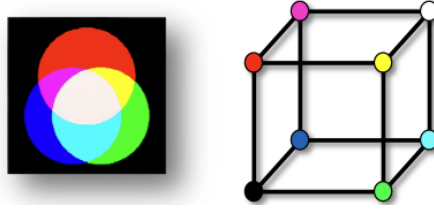


Figure 3: Addictive color model.

We can also define the opposite process, called *subtractive color model*. In this model, we start from a white image, and we subtract different amounts of cyan, magenta and yellow to it (the **primary colors**), creating a wide range of colors.

2.3.2 Problems of RGB

The RGB representation is not the optimal one. In fact, if we analyze how the human eye perceives colors, we can see that it is a lot more sensitive to some colors than others. For example, we are more sensitive to luminance than to color information, and less sensitive to color variations. This implies that a lot of information is redundant in the RGB representation, and other representations may be more effective.

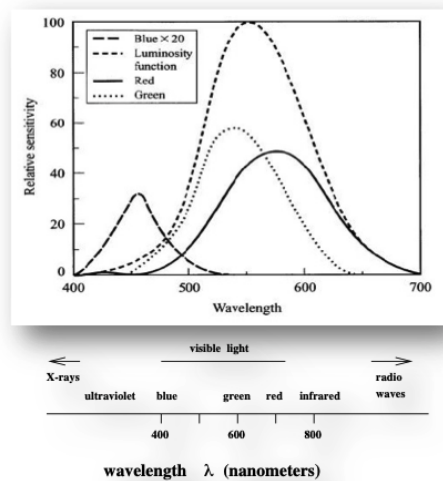


Figure 4: Sensitivity of the human eye to different colors.

Also, if we take an RGB image and separate its three components, we can see that they are highly correlated, meaning that they contain a lot of redundant information. This is also not ideal for image processing tasks.

2.3.3 YCbCr

The **YCbCr** representation is a color space that separates the luminance (Y) from the color information (Cb and Cr). This representation is more suitable for image compression (used in JPEG and MPEG standards) and transmission, as it allows to reduce the amount information, without losing too much quality to the human eye.

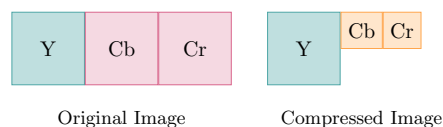


Figure 5: YCbCr compression example.

As we can see from the figure, by down-sampling the Cb and Cr components by a factor of 2, we can reduce the size of the image in half, without any noticeable difference in quality to the human eye.

2.3.4 HSV

The **HSV** (Hue, Saturation, Value) representation is another color space that separates the hue (H) from the saturation (S) and value (V). This representation is more suitable for computer graphics and image processing.

2.3.5 Bayer Pattern

In the acquisition process, the sensor of a camera must be able to capture the light information for all the three components in order to create a color image. The perfect solution would be to have three different sensors, one for each color component, but this is not practical and too expensive.

The most common solution is to use a **Bayer pattern**, which is a mosaic of color filters that are placed on top of the sensor. They are arranged in a specific pattern, to correctly exploit the sensitivity of the human eye to different colors. The pattern is composed as follows:

- 50% of the pixels are green, to capture the luminance information, which is the most important for the human eye.
- 25% of the pixels are red, to capture the red color information.
- 25% of the pixels are blue, to capture the blue color information.

This pattern allows to capture the color information in a single shot, without the need for multiple sensors, and it is widely used in digital cameras.

2.3.6 Quantization

Once we have the discrete signal, we need to quantize it, meaning that we need to assign a finite number of values to the pixel values. The most common quantization is to use 8 bits per pixel (bpp), which allows to represent 256 different values for each channel.

This is a good compromise between the quality of the image and the amount of data that we need to store and transmit, while also being handy for processing, as each pixel value can be represented by only one byte of memory. Using less than 8 bpp would result in a loss of quality, introducing the problem of **contouring**.

After quantization, we can represent the pixel values as integers in the range $[0, 255]$, where 0 represents black and 255 represents the maximum intensity of the color. For colored images, we can represent the pixel values as a tuple of three integers, one for each channel (R, G, B), in the range $[0, 255]$.

2.4 Videos

While static images provide information about static scenes, they lack motion information such as temporal evolution and dynamic interactions. Videos, on the other hand, capture a sequence of images over time, providing a richer representation of the world.

A video can be seen as a sequence of 2D images, that represent the projection of a **moving** 3D scene onto the video camera sensor. Usually videos have a lower resolution than images, as they would be too heavy to store and transmit otherwise. Each second of video is composed by a certain number of frames (typically 24, 30 or 60), and each frame is a 2D image. For videos lasting several minutes, the amount of data can be huge. Also, the information carried by videos is mostly given by the motion, and not by the color information.

2.4.1 Important Features

Once the images are acquired, we can extract some important features from them. Usually we are interested in:

- **Color** information and its distribution in the image. Described by the histogram of the image;
- **Shape** information, given by the contours and edges in the image. Described by the edges of the image.

If we take in consideration videos, we are interested how the two previous features evolve over time. This has many challenges

2.5 Histogram

A histogram is a simple way to describe the color distribution of an image. It can be seen as a probability distribution of the pixel values in the image. For a gray-scale image, the histogram is a 1D array of 256 values, where each value represents the number of pixels in the image that have the corresponding intensity value.

For an $M \times N$ grayscale image I , we have the following:

- The sum over all the histogram values is equal to the total number of pixels in the image:

$$\sum_{i=0}^{255} H(i) = M \times N$$

- We can represent the histogram as a probability distribution by normalizing it:

$$hist(i) = \frac{\#pixels : I(c, r) = i}{M \times N}$$

This can obviously be extended to color images, by computing the histogram for each channel separately.

2.5.1 Information from the histogram

The histogram can give us some information about the image, such as:

- The **mean** intensity of the image, giving us an idea of how bright or dark the image is;
- The **distribution** of colors in the image.

This can be seen as a signature of the image, and it can be used for very different tasks.

2.6 Simple Operations on the Histogram

Given the histogram of an image, we can apply some simple operations to it, in order to enhance the image or to extract some information from it.

2.6.1 Stretching

One of the most common operations that we can do on an image is **stretching**. This is a simple operation that allows to enhance the contrast of the image, by stretching the histogram to cover the entire range of pixel values.

To do this, we usually apply a piece-wise linear transformation to the pixel values. We define two thresholds, t_1 and t_2 , and we apply the following transformation to the pixel values:

- If $I(c, r) < t_1$, we set $I'(c, r) = 0$. This means that all the pixel values below t_1 are set to black.
- If $I(c, r) > t_2$, we set $I'(c, r) = 255$. This means that all the pixel values above t_2 are set to white.
- If $t_1 \leq I(c, r) \leq t_2$, we create a linear transformation that maps the range $[t_1, t_2]$ to the range $[0, 255]$:

The first and the second operations are called **clipping**, while the third one is called **scaling**.

2.6.2 Equalization

Ideally, we would like to have a histogram that is as flat as possible, meaning that all the pixel values are equally represented in the image. This can be achieved by applying a transformation to the image called **equalization**. This transformation is defined as follows:

- **Cumulative histogram:** We first compute the cumulative histogram of the image, which is defined as:

$$CHist_I(i) = \sum_{k=0}^i hist(k)$$

- **Equalization transformation:** We then apply the following transformation to the pixel values:

$$hist_{equalized}(i) = \frac{CHist_I(i) - CHist_I(0)}{M \times N} \times 255$$

2.7 Edge Extraction

Edge extraction is a fundamental operation in image processing, as it allows to extract the shape information from the image. An edge can be seen as a **sharp change** in the brightness of the image.

The process of edge extraction can be seen as a two-step process:

- **Edge detection:** determine the direction and the strength of edges for each pixel in the image. This can be done using different techniques, such as gradient-based methods or Laplacian-based methods.
- **Binarization:** Once we have the edge strength for each pixel, we need to apply a threshold to it, in order to determine which pixels are edges and which are not (binary image of edges).

2.7.1 Gradient based approach

In the gradient-based approach, the goal is to find maxima and minima pixels in the image, along a certain direction. This can be formalized as:

$$\frac{\partial f}{\partial r} = \frac{\partial f}{\partial x} \frac{\partial x}{\partial r} + \frac{\partial f}{\partial y} \frac{\partial y}{\partial r} = f_x \cos(\theta) + f_y \sin(\theta) \quad (2.1)$$

Where f is the image, r is the direction of the edge, and θ is the angle of the edge with respect to the x-axis. We can then derivate it again with respect to θ to find the maxima and minima directions:

$$\frac{\partial}{\partial \theta} \left(\frac{\partial f}{\partial r} \right) = -f_x \sin(\theta_g) + f_y \cos(\theta_g) = 0 \implies \theta_g = \arctan \left(\frac{f_y}{f_x} \right) \quad (2.2)$$

Where θ_g is the direction of the edge. We can then compute the edge strength as the magnitude of the gradient:

$$\sqrt{f_x^2 + f_y^2} \quad (2.3)$$

2.7.2 Practical Example - Sobel Operators

In practice, **FIR filters** are typically adopted to compute the gradient of the image, as they are computationally efficient and can be easily implemented using convolution.

The most common FIR filters for edge detection are the **Sobel operators**, which are defined as follows:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (2.4)$$

For each pixel in the image, we apply the two filters to it and compute the edge strength as the magnitude of the gradient:

$$D = \sqrt{D_x^2 + D_y^2} \quad (2.5)$$

Where D_x and D_y are the results of the convolution of the image with the G_x and G_y filters, respectively.

2.7.3 Convolution

In order to apply the filters to the image, we need to perform a convolution operation between an image f and a filter h . In 2D, the convolution is defined as follows:

$$g(x, y) = (f * h)(x, y) = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} f(i, j)h(x - i, y - j) \quad (2.6)$$

In simpler words, we first flip the filter h 180 degrees counter-clockwise, and then we slide it over the image f , computing the sum of the products of the overlapping pixels at each position.

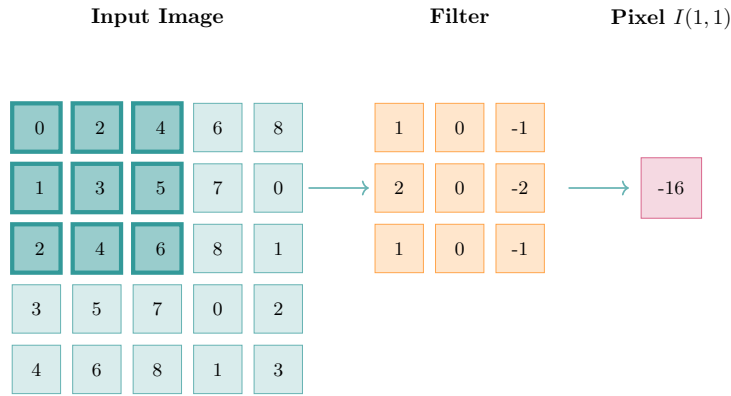


Figure 6: Example of convolution between an image and a filter.

It's important to note that the output of the convolution operation must be stored in a new image, as we need to use the original image for the next pixels.

2.8 Smoothing / Enhancement

Another type of operation that we can apply to an image is **smoothing**.

2.8.1 Low-pass filter

The idea of the low-pass filter is to remove the high-frequency components of the image, consisting of the edges and the fine details, and leaving only the low frequency components, which are typically associated with the smooth regions of the image.

For a 5×5 low-pass filter, we can define it as follows:

$$I'(c, r) = \frac{1}{25} \sum_{i=-2}^2 \sum_{j=-2}^2 I(c + i, r + j) \quad (2.7)$$

Each kernel value is equal to $\frac{1}{M \times N}$, where M and N are the dimensions of the kernel. This is simply the average of the pixel values in the neighborhood of the pixel we are processing.

Obviously, the bigger the kernel, the stronger the smoothing effect. However, we have to take in consideration that the pixel at the edge of the image cannot be processed. For this reason, making the kernel bigger will result in a bigger portion of the image that cannot be processed.

If wanted, we could also design a **high-pass filter** or a **band-pass filter**, by combination of filters.

2.8.2 Gaussian filter

The Gaussian filter is a low-pass filter that is defined as follows:

$$G(x, y) = ce^{-\frac{x^2+y^2}{2\sigma^2}} \quad (2.8)$$

Where c is a normalization constant to ensure integer values for the kernel, and σ is the standard deviation of the Gaussian distribution, which controls the amount of smoothing. This is an **isotropic** filter, meaning that it has the same values in all directions. This implies that, when we apply it to an image, there's no need to compute the rotation of the filter.

The effects of this filter are similar to the ones of the low-pass filter, but instead of taking the average of the pixel values in the neighborhood, it takes a weighted average, where the weights are inversely proportional to the distance from the center of the kernel. This means that the pixels closer to the center of the kernel will have a stronger influence on the output pixel value, while the pixels farther away will have a weaker influence.

2.8.3 Median filtering

The median filter is a non-linear filter that is used to remove noise from an image, especially for spike noise. If we have noise in the image, applying a low-pass or a Gaussian filter will not be effective, as the mean operator is very sensitive to outliers. On the other hand, the median operator is known to be more robust to outliers, thus it is more effective in removing noise from the image.

Instead of taking the average of the pixel values in the neighborhood, the median filter takes the median value of the pixel values in the neighborhood. The

elements of the neighborhood are sorted in ascending order, and the value in the middle is taken as the output pixel value.

3 Morphology Filters

Morphology operators are a set of non-linear operations used to extract information about the shape of objects in an image. They are typically used for different tasks, such as:

- **Shape fitting:** Given a shape, check whether it fits inside an object in the image;
- **Shape extraction:** Given a shape, extract all the objects in the image that have that shape;
- **Shape analysis:** Check whether a picture has a certain shape;

Those operators are typically applied to **binary images**, where the pixel values are either 0 or 1, given a **Binary Structuring Element** (BSE), which represents the shape that we want to fit, extract or analyze.

Common operators are:

- **Erosion:** reduce the area of a certain shape in the image;
- **Dilation:** increase the area of a certain shape in the image;
- **Opening:** remove small portions of the image, close to the boundary of relevant areas;
- **Closing:** fill small holes in the image and makes region boundaries smoother.

Opening and closing are combinations of erosion and dilation.

Dilation

The structuring element S is swept over the binary image B . As the origin of the structuring element touches a pixel in the image with a value of 1, all pixels in the image that are covered by the structuring element are ORed with the value of B . This basically corresponds to sliding the structuring element over the border of the shape in the image, and adding pixels to it, thus increasing its area.

More formally, the dilation of the foreground of a binary image B , with a structuring element S is defined as follows:

$$B \oplus S = \bigcup_{b \in B} S_b \quad (3.1)$$

These operations are usually used to simplify recognition and tracking of objects in an image.

Erosion

Again, the structuring element S is swept over the binary image B . For each position where the structuring element is fully contained in the foreground of the image, the pixel in the output image are ORed with the value of B .

Closing and Opening

Closing and opening are combinations of dilation and erosion. They are defined as:

- **Closing:** first we apply **dilation** to the image, and then we apply **erosion** to the result;
- **Opening:** first we apply **erosion** to the image, and then we apply **dilation** to the result.

4 Models

To represent the real world using equations, we first need to define a model. A model is a **mathematical approximation** of the real world, which allows describing events and phenomena using parametric representations. The parameters of the model can be used for processing purposes.

4.1 Pinhole Camera Model

The first model we will discuss is the **pinhole camera model**, which relates to how a camera captures an image. It assumes that an object in the real world emits light rays that pass through a small hole (the pinhole) and project onto an image plane.

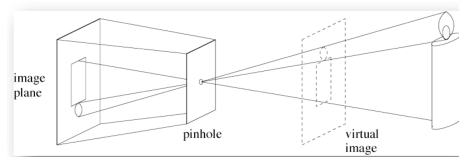


Figure 7: Pinhole Camera Model

We want to make sure that only a single ray of light from each point passes through the pinhole (ideally, it should be a point), which allows us to create a clear image on the image plane. Bigger holes would allow multiple rays from the same point to pass through, resulting in a blurry image.

On the image plane, the rays of light from the object are projected, creating an image which is flipped upside down and its size is determined by the distance between the object and the pinhole and the distance between the pinhole and the image plane (also known as the **focal length f**). The relationship between the object size, the image size, and the distances can be described using similar triangles, as follows:

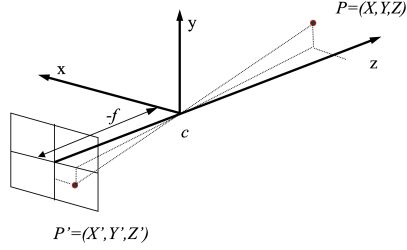


Figure 8: Projection of the point P onto the image plane

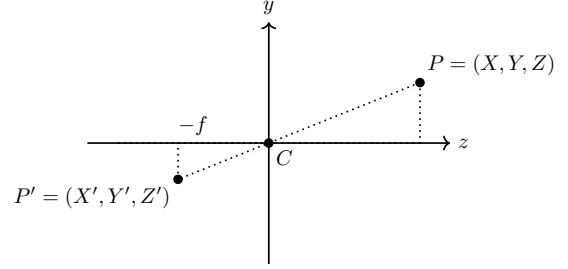


Figure 9: Projection of the point P onto the image plane (2D representation)

As shown in the figure, the point P in the real world is projected onto the image plane as P' . The coordinates of P' can be obtained using the geometric properties of the pinhole camera model. In particular, the triangles formed by the points P , C (the pinhole), and P' are similar, which means that the ratios between the corresponding sides are equal.

This leads to the following relations:

$$\begin{cases} Z' = -f \\ \frac{Z}{Z'} = \frac{X}{X'} \\ \frac{Z}{Z'} = \frac{Y}{Y'} \end{cases}$$

Substituting $Z' = -f$ and rearranging the equations, we obtain

$$\begin{aligned} X' &= -f \frac{X}{Z} \\ Y' &= -f \frac{Y}{Z} \end{aligned}$$

Therefore, the projection of the 3D point $P = (X, Y, Z)$ onto the image plane is given by

$$P' = \left(-f \frac{X}{Z}, -f \frac{Y}{Z}, -f \right).$$

4.1.1 Features of the model

The pinhole camera model has several important features:

- Further objects appear smaller in the image, which is a consequence of the projection process. It is not able to correctly capture the depth information of the scene;

- Parallel lines in the real world appear to converge towards a single point in the image, known as the **vanishing point**. The line connecting the vanishing is called the **horizon line**. Vertical lines in the real world will still appear vertical in the image, being perpendicular to the horizon line;
- The image is inverted (upside down) due to the way light rays pass through the pinhole;
- The model assumes that the pinhole is infinitely small, which is obviously an idealization. This would also require an infinite amount of light to pass through, which is not practical.

4.2 Projection

By the time we capture an image, we have already projected the 3D world onto a 2D plane (the image plane). Considering also the time dimension, we can formalize the projection process as a mapping from a 4D space to a 3D space:

$$f : \mathbb{R}^4 \rightarrow \mathbb{R}^3$$

$$(X, Y, Z, t) \mapsto (x, y, t)$$

We can represent two types of projections: **perspective projection** and **orthographic projection**.

4.2.1 Perspective Projection

Similar to the pinhole camera model, the perspective projection captures the way objects appear smaller as they are further away from the camera. The only difference is that now the image plane is placed on the origin of the coordinate system, while the pinhole (lens center) is placed between the image plane and the object, at a distance equal to the focal length f .

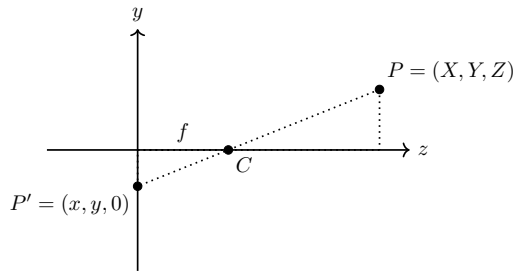


Figure 10: Perspective Projection

The triangles formed by the points P , C (the pinhole), and P' are again similar, which leads to the following relations:

$$\begin{cases} Z' = 0 \\ \frac{f}{Z-f} = \frac{x}{X} \\ \frac{f}{Z-f} = \frac{y}{Y} \end{cases}$$

Substituting $Z' = 0$ and rearranging the equations, we obtain

$$\begin{aligned} x &= f \frac{X}{Z-f} \\ y &= f \frac{Y}{Z-f} \end{aligned}$$

Therefore, the projection of the 3D point $P = (X, Y, Z)$ onto the image plane is given by

$$P' = \left(f \frac{X}{Z-f}, f \frac{Y}{Z-f}, 0 \right).$$

We can notice that knowing the coordinates of the projection P' and the focal length f , we cannot recover the original coordinates of the point P , since we have two equations and three unknowns. This is a consequence of the fact that we have lost the depth information during the projection process, which is a fundamental limitation of the perspective projection model. In case two cameras are used to capture the same scene from different viewpoints, we can use the information from both images to recover the depth information of the scene.

4.2.2 Simplified Perspective Projection

The previous model still suffer from the limitation of the pinhole camera model, that flips the image upside down. We can easily fix this by placing a virtual image plane between the pinhole and the object, at a distance equal to the focal length f .

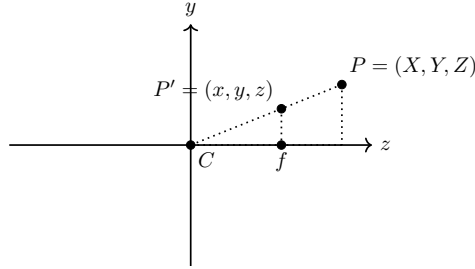


Figure 11: Simplified Perspective Projection.

The final equations for the projection of the 3D point $P = (X, Y, Z)$ onto the image plane are the same as in the perspective projection model, removing the term $-f$ from the denominator:

$$P' = \left(f \frac{X}{Z}, f \frac{Y}{Z}, 0 \right).$$

It's important to note that this model is a simplification of the perspective projection model, which is only valid when $f \ll Z$, meaning that the focal length is much smaller than the distance between the object and the camera. In the real world, this is often the case, since the focal length of a camera is typically a few millimeters, while the distance between the camera and the objects in the scene is usually much larger.

4.2.3 Orthographic Projection

In the orthographic projection, we assume that all the rays of light from the object are parallel to each other and perpendicular to the image plane. This means that the size of the object in the image does not change with distance, and there is no perspective distortion.

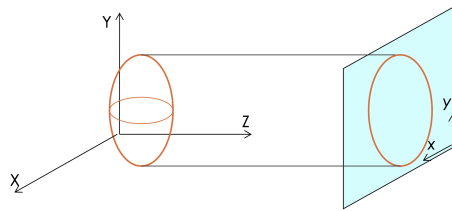


Figure 12: Orthographic Projection model.

This assumption is valid when the distance between the camera and the objects in the scene is much larger than the depth of the scene, which is often the case in applications such as satellite imaging or architectural drawings.

The projection of a 3D point $P = (X, Y, Z)$ onto the image plane in the orthographic projection model is given by

$$P' = (X, Y, 0)$$

This could also be rewritten in matrix form as follows:

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

We can see this as a special case of the perspective projection model, where the focal length f is infinite, which means that the rays of light are parallel to each other and perpendicular to the image plane.

4.3 Illumination Model

Illumination plays a very critical role in Computer Vision. The way light interacts with objects in the scene can significantly affect the appearance of the objects in the image, and thus, the performance of computer vision algorithms.

When a light source hits an object, it can be **absorbed**, **reflected**, or **transmitted** through the object.

This is quite difficult to model from the camera's perspective, since we perceive objects based on the light that is reflected from them.

4.3.1 Reflection

Reflection can be of two types:

- **Specular:** more energy is concentrated in the light source direction;
- **Diffuse:** constant energy in all directions. The position of the observer is irrelevant,

4.3.2 Surface Properties

The type of reflection depends on the surface properties of the object, such as its roughness, color, and texture. Some of them reflect light uniformly in all directions (matte surfaces), while others reflect light in a more concentrated manner (glossy surfaces).

It would be ideal if all objects had matte surfaces, since they would be easier to model and analyze, while glossy surfaces can create artifacts in the image, which can make it difficult to extract useful information from the image.

Lambertian Surfaces

Let's first suppose that we have only one light source which is far away from the scene, so that we can assume that all the rays of light emitted from the light source are parallel to each other.

For each surface point the light is irradiated considering the cosine of the angle between the **surface normal** and the **light direction**, which is known as the **Lambert's cosine law**. This means that the intensity of the light reflected from a surface point is proportional to the **cosine** of the **angle** between the surface normal and the light direction. Letting i be the intensity of the reflected light, s be the intensity of the light source, n be the surface normal, we can write the Lambert's cosine law as follows:

$$i = s \cdot n$$

This approximation is valid only for **diffuse reflection**, where the surface reflects light uniformly in all directions. This is not a realistic model for all surfaces, but it still provides a useful approximation which can be turned into a parametric model.

Lambertian Reflectance Model

The Lambertian reflectance model is a simple model that describes the way light is reflected from a surface point, assuming that the surface is a Lambertian surface. The model can be expressed as follows:

$$i = \rho s \cdot n \quad (4.1)$$

which can be computed as:

$$i = \rho \|n\| \|s\| \cos(\alpha)$$

where ρ is the albedo of the surface, which represents the fraction of light that is reflected from the surface.

An element is not visible if $n \cdot s \leq 0$, which means that the surface normal is pointing away from the light source, and thus, the surface point is in shadow.

4.4 Lenses

Up until now, we have assumed that our camera were pinhole cameras, which is not the case in the real world. In reality, cameras use **lenses** to focus light onto the image plane. This means that the objects in the scene are **on focus** if the distance between the camera and the image plane obeys the textitthin lens equation.

4.4.1 Thin Lens Equation

Using a lens instead of a pinhole means that a wide set of rays of light from the same point P are merged together to the same point P' on the image plane. The intensity of each ray of light is summed together, which allows us to capture more light and thus, create a brighter image.

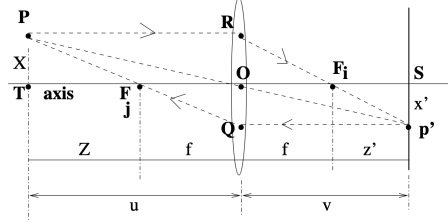


Figure 13: Thin Lens model.

Supposing that we have an object at distance u from the lens, and an image plane at distance v from the lens. Considering the similar triangles formed by the points P, R, O and p', S, O , we can derive that:

$$\frac{X'}{Z' + f} = \frac{X}{Z + f}$$

Knowing that $X = f \frac{x'}{Z'}$, we can substitute this into the previous equation to obtain:

$$\frac{X'}{Z' + f} = f \frac{Z'}{Z'(Z + f)}$$

Rearranging the equation, we get:

$$X' = f \frac{X'(Z + f)}{Z'(Z + f)}$$

We can simplify the X' term on both sides of the equation, which leads to:

$$Z'(f + Z) = fZ + f^2$$

Where $Z = u - f$ and $Z' = v - f$. Substituting these values into the previous equation, we end up with the thin lens equation:

$$\frac{1}{f} = \frac{1}{u} + \frac{1}{v} \quad (4.2)$$

This equation describes the relationship between the focal length of the lens, the distance of the object from the lens, and the distance of the image plane from the lens. We can notice that if the object goes to infinity ($u \rightarrow \infty$), then the image plane is located at the focal length of the lens ($v = f$).

4.4.2 Focus

If we move the image plane (changing v), the point P' will be projected to a different point on the image plane, which means that the objects in the scene will be out of focus. On the other hand, if P is moved, then u also changes, which means that the image plane will need to be moved in order to keep the objects in focus.

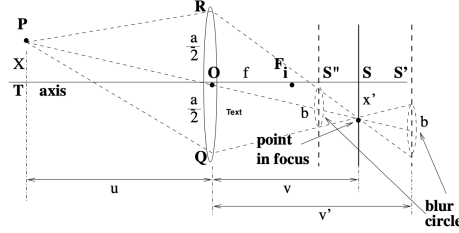


Figure 14: Focus and out of focus objects.

In these cases, instead of a single point P' on the image plane, we will have a **blur circle** of diameter b , which is the area on the image plane where the rays of light from the same point P are projected when they are out of focus.

When capturing an image, we first need to define the acceptable level of blur, which is determined by the size of the blur circle b . Once this is defined, we can use the thin lens equation to determine the range of distances from the camera where objects will be in focus, which is known as the **depth of field**.

Using the thin lens equation, we can derive the following formulation for the depth of field, which describes the range of distance for the image plane v' where the objects will be in focus, in terms of the object distance u , the focal length f , and the acceptable blur circle diameter b :

$$\begin{cases} v' = \frac{a+b}{a}v \\ v'' = \frac{a-b}{a}v \end{cases}$$

The depth of field is then given by the difference between v' and v'' .

Given this formulation and the value of the acceptable blur circle diameter b , we can determine the range of distances from the camera where objects will be in focus. This is important for computer vision applications, to ensure that the objects of interest in the scene are captured in focus. It is defined as:

$$\begin{cases} u_n = \frac{u(a+b)}{a + \frac{bu}{f}} \\ u_r = \frac{u(a-b)}{a - \frac{bu}{f}} \end{cases}$$

We can notice that:

- If f becomes smaller u_r becomes larger, which means that the object can be further away from the camera and still be in focus;
- If $u \rightarrow \infty$ then rays of light from the object are parallel to each other, which means that the image plane is located at the focal length of the lens ($v = f$), and thus, all objects will be in focus.

4.4.3 Autofocus

Modern cameras use autofocus systems to automatically adjust the position of the image plane in order to keep the objects in focus. We can distinguish between two types of autofocus systems:

- **Active autofocus:** the camera emits a signal (e.g., infrared light) and measures the time it takes for the signal to bounce back from the objects in the scene, which allows it to estimate the distance of the objects and adjust the focus accordingly. This method faces problems in case of far objects, obstacles or glossy surfaces, which can reflect the signal in unexpected ways.
- **Passive autofocus:** the camera analyzes the image itself to determine the level of focus, and adjusts the position of the image plane accordingly. Usually, the camera will try to analyze the edges in the image, since they are more affected by blur.

High quality cameras use a combination of both active and passive autofocus systems to ensure that the objects in the scene are captured in focus, regardless of the distance or lighting conditions.

4.4.4 Resolution, Blur and Resolving Power

A camera with a $N \times M$ pixel sensor can detect at most $\frac{N}{2}$ horizontal lines (one pixel must be left between two lines to distinguish them). If the blur is larger than the distance between two lines, then the camera will not be able to distinguish them, leading to a loss of detail in the image.

We define the **resolving power** of a camera as the maximum number of lines per unit distance that the camera can distinguish, which is given by the formula:

$$R_p = \frac{1}{2\Delta} \quad (4.3)$$

where Δ is the spacing between two lines that the camera can distinguish.

Example Consider a camera sensor of size $36mm \times 24mm$, with a resolution of 4000×2500 pixels. We can calculate the spacing between two lines that the camera can distinguish as follows:

$$R_p = \frac{1}{2\Delta} \text{ where } \Delta = \frac{\text{sensor size}}{\text{resolution}}$$

Calculating Δ for the vertical direction:

$$\Delta = \frac{24mm}{2500} \approx 0.01mm$$

Thus, the resolving power is:

$$R_p = \frac{1}{2 \times 0.01mm} = 50 \text{ lines/mm}$$

This means that the camera can distinguish up to 50 lines per millimeter, which is a measure of the level of detail that the camera can capture in the image.

Assuming we now want to capture an object in the real world. In particular, we have a focal length of $f = 50mm$ and a distance of the object from the camera of $u = 5000mm$. We want to compute the minimum size of the object that can be distinguished by the camera.

First, we can notice that the rays of light from the object are parallel to each other, which means that rays of light coming from adjacent lines have an angle θ separating them. In order to distinguish between two lines, the spacing between pixel lines must be greater than 2Δ . We can write that:

$$2\Delta = f \sin(\theta) \approx f\theta$$

Given the focal length f , we can calculate the angle θ as follows:

$$\theta = \frac{2\Delta}{f} = \frac{2\Delta}{50mm} = 0.0004 \text{ radians}$$

Now, we can calculate the minimum size of the object that can be distinguished by the camera using the formula:

$$\text{object size} = u \tan(\theta) \approx u\theta$$

Substituting the values, we get:

$$\text{object size} = 5000mm \times 0.0004 = 2mm$$

This means that the camera can distinguish objects that are at least 2mm in size at a distance of 5000mm, given the focal length and resolution of the camera.

4.4.5 Typical Issues

There are several issues that can arise when using lenses, which can affect the quality of the image and the performance of computer vision algorithms. Some of these issues include:

- **Barrel distortion:** caused by wide-angle lenses, where straight lines appear to curve outward from the center of the image;
- **Chromatic aberration:** typically related to bad lenses, where different wavelengths are threatened differently, resulting in color fringing around the edges of objects in the image;
- **Vignetting:** caused by cheap or dirty lenses, where the corners of the image appear darker than the center due to light falloff.

5 Motion detection

In this section we will discuss the analysis of a scene, not only in terms of a static image, but also in terms of temporal evolution of it. **Motion** is one of the most important features of a scene, and we are usually interested in detecting:

- **Moving objects**, which are the objects that are moving in the scene;
- **Camera motion**, which is the motion of the camera itself.

Motion can be formally defined as the change of position of an object in a scene over time. In the 3D case, it can be represented as:

$$D(\mathbf{X}, t_1, t_2) = \mathbf{X}' - \mathbf{X} = [\Delta x, \Delta y, \Delta z] \quad (5.1)$$

When projected into the 2D image plane we get:

$$d(\mathbf{x}, t_1, t_2) = \mathbf{x}' - \mathbf{x} = [\Delta x, \Delta y] \quad (5.2)$$

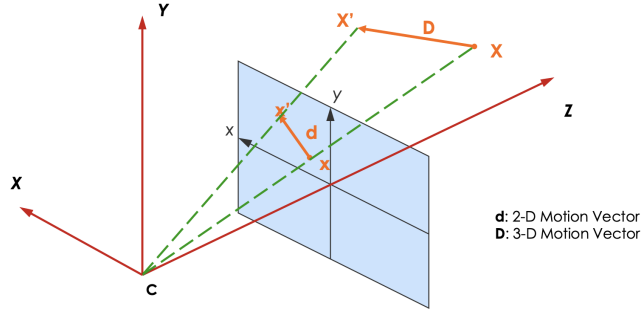


Figure 15: Projection of movement from 3D to 2D.

From this, if we consider a single point, we can derive that the displacement we perceive in the image plane can be quite different from the actual 3D displacement, due to the projection of the 3D scene onto the 2D image plane.

The final goal of the analysis of motion is to be able to extract the **motion field**, which is the set of all the displacements of all the points in the scene, understand if it really corresponds to the actual motion of the objects in the scene, and understand if it was caused by the motion of the camera itself or by the motion of the objects in the scene.



Figure 16: Example of motion detection.

An example In this example, we can clearly identify three main components of the motion field:

- The motion of the face, which moved from left to right between the two frames. In fact, in the **motion field**, we can see the arrows pointing from left, as signaling that the "point" of frame 2 is to the right of the corresponding "point" in frame 1.
- Static section, corresponding to the background, which is not moving, and therefore has no motion field. If arrows were present, they would represent the motion of the camera, as the background of this example is static;
- The motion of the head. Since the safety helmet is mostly flat, the algorithm is not able to correctly identify the motion of the head. In the motion field, this results in a "noisy" motion field, with arrows pointing in different directions, and not corresponding to the actual motion of the head.

Applications Motion detection is a fundamental task in computer vision, and it has many applications, such as:

- **Detection of moving objects**, which is useful for surveillance, traffic monitoring, and many other applications;
- **Segmentation of moving objects**;
- **Recognition**, we could filter different objects based on their motion (e.g., we could recognize a car from a pedestrian based on their motion);
- **Filtering**;
- **Compression**, in case of videos, the motion component can be used to compress the video.

In general, it is used to detect changes of the image intensity, in the scene over time.

5.1 Problems

The motion detection task is quite challenging, and the solution might be **not unique**.

In fact, the motion is based on the **velocity** of the points in the scene, over two frames. This might not reflect the actual motion of the objects in the scene, as it might also be caused by **camera motion** or by **illumination changes**.

5.1.1 Real vs Apparent movement

Also, being the motion perceived by changes in the scene, we could have two possible situations for which the motion field is not reflecting the actual motion of the objects in the scene:

- **Constant illumination**, in which a sphere is moving in the scene (rotating around its own axis), but the illumination is constant. In this case, the motion field will be zero, as there are no changes in the image intensity, even though the sphere is moving.
- **Changing illumination**, in which the sphere is not moving, but the illumination source rotates around the sphere. In this case, the motion field will be non-zero, as there are changes in the image intensity, even though the sphere is not moving.

5.1.2 Occlusion

Another non-trivial problem is the occlusion. This problem occurs when an object in the background is partially or completely occluded by an object in the foreground. This can cause problems in the motion detection, as a static background might be perceived as moving, due to the occlusion of the foreground object.

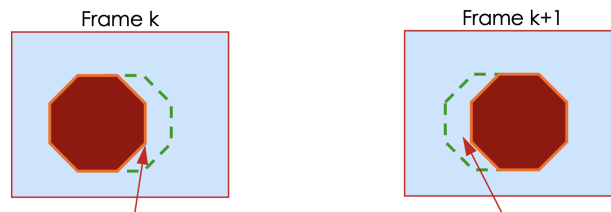


Figure 17: Example of occlusion.

5.1.3 Aperture

Not all motion can be detected by analyzing the changes in the image intensity. In fact, only displacements of the borders of the objects can be detected, while the motion of the internal points of the objects cannot be detected.

Also, borders can only detect the motion in the direction perpendicular to the border itself (in the direction of the intensity gradient), while for angles, we can detect motion in both directions.

5.2 Optical Flow

The motion detection we have described so far can be formalized through the concept of **optical flow**.

First, we define the assumption taken in the optical flow:

- **Brightness constancy:** the object illumination does not change over time;
- **Small motion:** the motion between two frames is small enough;
- **Spatial coherence:** each point $[x, y]$ in the image plane is shifted to a new position $[x + \Delta x, y + \Delta y]$. This imposes that the motion is only a translation, and that there is no rotation or scaling.

This is usually not true in real scenarios, but it is a good approximation for small motions.

5.2.1 Optical Flow Function

The optical flow function is defined as:

$$\psi(x + \Delta x, y + \Delta y, t + \Delta t) = \psi(x, y, t) \quad (5.3)$$

This means that the intensity of the point $[x, y]$ at time t is the same as the intensity of the point $[x + \Delta x, y + \Delta y]$ at time $t + \Delta t$, as long as the motion is small enough. This is the brightness constancy assumption.

The Taylor expansion states that for a function $f(x + \Delta x)$, we can approximate it as:

$$f(x + \Delta x) \approx f(x) + \frac{\partial f}{\partial x} \Delta x$$

By applying the Taylor expansion to the left-hand side of the equation 5.3, we get:

$$\psi(x + \Delta x, y + \Delta y, t + \Delta t) \approx \psi(x, y, t) + \frac{\partial \psi}{\partial x} \Delta x + \frac{\partial \psi}{\partial y} \Delta y + \frac{\partial \psi}{\partial t} \Delta t$$

By substituting this into the equation 5.3, we get:

$$\frac{\partial\psi}{\partial x}\Delta x + \frac{\partial\psi}{\partial y}\Delta y + \frac{\partial\psi}{\partial t}\Delta t = 0$$

By dividing both sides by Δt , we get:

$$\frac{\partial\psi}{\partial x} \frac{\Delta x}{\Delta t} + \frac{\partial\psi}{\partial y} \frac{\Delta y}{\Delta t} + \frac{\partial\psi}{\partial t} = 0$$

By defining the velocity of the point as $v_x = \frac{\Delta x}{\Delta t}$ and $v_y = \frac{\Delta y}{\Delta t}$, we get the optical flow equation:

$$\frac{\partial\psi}{\partial x}v_x + \frac{\partial\psi}{\partial y}v_y + \frac{\partial\psi}{\partial t} = 0 \quad (5.4)$$

This implies that the variation of the intensity of a point in the image plane is related to the velocity of the point in the image plane.

If we decompose the velocity into its components (normal and tangential with respect to the surface of the object), we can see that only the normal component of the velocity can be detected, since the tangential component does not cause any change in the intensity of the point. This is the aperture problem we have discussed before.

5.3 Motion Detection in practice

There are two main approaches to motion detection in practice:

- **Intensity-based methods**, which are based on the analysis of the changes in the image intensity over time;
- **Feature-based methods**: based on the analysis of the changes in the features of the objects in the scene over time. This requires additional steps of feature extraction and matching, but it can be more robust to changes in the illumination and to occlusions.

5.3.1 Change Detection

The change detection is a simple approach to motion detection. It takes as input two frames of a video, taken at different time instants, and it computes the difference between the two frames. This can be done by simply following the following steps:

- **Subtraction**: we compute the difference between the two frames, pixel by pixel. The resulting image will have high intensity values in the areas where there is a change in the intensity, and low intensity values in the areas where there is no change.

- **Thresholding:** we apply a threshold to the resulting image, to obtain a binary image. The pixels with intensity values above the threshold will be set to 1, and the pixels with intensity values below the threshold will be set to 0.

From a computational point of view, this approach is very simple and fast. We can formalize it as:

- **Frame difference:** corresponds to the subtraction of two images $I_K - I_{K-1}$. We can define the **frame differencing operation** as:

$$FD_{K,K-1}(x, y) = s(x, y, k) - s(x, y, k - 1) \quad (5.5)$$

If FD is not null, then a change has been detected in the pixel $[x, y]$;

- **Thresholding:** to filter out the noise, we can apply a threshold to the resulting image. We can define the **thresholding operation** as:

$$T(x, y) = \begin{cases} 1 & \text{if } FD_{K,K-1}(x, y) > \tau \\ 0 & \text{otherwise} \end{cases} \quad (5.6)$$

This algorithm allows to quickly adapts to changes in the backgrounds and to the stopping of the moving objects. It can detect the contours of the moving objects, but it is not able to detect the motion of the internal points of the objects, due to the aperture problem.

To resolve these problems, usually the time scale of the analysis is increased, by considering more distant frames. This allows to detect the motion of the internal points, but for fast moving objects, it can cause problems.

5.3.2 Background subtraction

The background subtraction is a more sophisticated approach to motion detection. It is based on the idea of modeling the background of the scene as a static image (acquired a priori).

The algorithm can be formalized as follows: This allows for a better representa-

Algorithm 1 Background subtraction algorithm

```

1:  $B = I(0)$ ; ▷ Initialize background model
2: for each frame  $I(t)$  do
3:    $D = |I(t) - B|$ ; ▷ Difference between current frame and background
4:    $M = Threshold(D)$ ; ▷ Apply thresholding to obtain the motion mask
5: end for

```

tion of the moving objects in the scene, as it can filter out the static background, but it has a few limitations, such as:

- **Modelling the background:** the background model might not be accurate, especially in case of dynamic backgrounds;
- **Changes in the background:** if a background object starts moving, both the real object and its "ghost" will be detected as moving objects;
- **Illumination changes:** if the illumination changes, the background model is not able to adapt to it as it is based on a static image;
- **Camera motion:** if the camera is moving, again the background model is not able to adapt to it.

5.3.3 Adaptive Background Subtraction

To overcome the limitations of the background subtraction, we can use an adaptive background subtraction approach. In this approach, the background model is updated over time, to adapt to changes in the background and to changes in the illumination. The update of the background model can be formalized as:

$$B_t = \alpha I(t) + (1 - \alpha)B_{t-1} \quad (5.7)$$

Where α is a parameter that controls the update rate of the background model:

- If α is close to 1, the background model will be updated quickly, going back to the **change detection** approach;
- If α is close to 0, the background model will be updated slowly, maintaining a more stable representation of the background.

5.3.4 Gaussian Average

Instead of relying on a single background model, we can use a Gaussian Model to represent the background. In this approach, we model the background as a Gaussian distribution, and we update the parameters of the distribution over time, with the same approach of the adaptive background subtraction.

We can formalize the Gaussian model as follows:

$$\mu_t = \alpha I_t + (1 - \alpha)\mu_{t-1} \quad (5.8)$$

This model can adapt to changes in the background and to changes in the illumination, by "absorbing" the changes in the background into the parameters of the Gaussian distribution. The speed of adaptation is controlled by the parameter α , as in the adaptive background subtraction approach.

This model allows recognizing the moving objects in the scene, even in case of dynamic backgrounds. We can instead say that a pixel belongs to the foreground if the intensity value of the pixel is far from the mean of the Gaussian distribution, by a certain threshold:

$$|I_t - \mu_t| > k\sigma_t \quad (5.9)$$

Where k is the threshold parameter, and σ_t is the standard deviation of the Gaussian distribution at time t .

Even though the model is constantly updated, usually an initialization phase is required, in which the background model is learned from a sequence of frames. This allows to have a more accurate representation of the background, and to reduce the number of false positives in the motion detection.

5.3.5 Improving the Gaussian Average

From the equation 5.8 of the Gaussian average, we can see that the update of the mean is based on both background and foreground pixels, with no distinction between them.

To avoid foreground pixels to affect the update of the background model, we can use a **selective update** approach, in which only the pixels that are classified as background are used to update the background model. This can be formalized as:

$$\mu_t = M\mu_{t-1} + (1 - M)(\alpha I_t + (1 - \alpha)\mu_{t-1}) \quad (5.10)$$

Where M is a binary mask that indicates which pixels are classified as background (1) and which pixels are classified as foreground (0). If the pixel is classified as foreground, the background model is not updated at all, while if the pixel is classified as background, the background model is updated as in the Gaussian average approach.

5.3.6 Mixture of Gaussian

There are cases in which a single Gaussian distribution is not enough to model the background. For example, in case of rain, snow, or waving trees, the background can have multiple modes, and a single Gaussian distribution is not able to capture all the variations in the background. These are known as multimodal backgrounds, and they require a more complex model to be able to capture all the variations in the background.

In this case, we can use a mixture of Gaussian distributions to model the background. In particular, we can model the background as a mixture of K Gaussian

distributions, where each distribution represents a different mode of the background. More formally, we can define the mixture of Gaussian model as:

$$P(X_t) = \sum_{i=1}^K w_{i,t} \mathcal{N}(X_t | \mu_i, \Sigma_i) \quad (5.11)$$

Where:

- $P(X_t)$ is the probability of the pixel intensity X_t at time t ;
- $w_{i,t}$ is the weight of the i -th Gaussian distribution at time t , which represents the importance of the i -th mode of the background at time t ;
- $\mathcal{N}(X_t | \mu_i, \Sigma_i)$ is the Gaussian distribution with mean μ_i and covariance Σ_i .

By assuming that each color channel is independent, we can simplify the model by using a diagonal covariance matrix, which reduces the number of parameters to be estimated.

Identify each Distribution Once we have defined the K Gaussian distributions, we need to rank them based on their weights $w_{i,t}$, amplitude and standard deviation, to determine which distributions represent the background and which distributions represent the foreground. Usually, the distributions with the highest weights and lowest standard deviation are considered as background distributions, while the distributions with the lowest weights and highest standard deviation are considered as foreground distributions. Formally, the first B distributions that satisfy the following condition are considered as background distributions:

$$\sum_{i=1}^B w_{i,t} > T \quad (5.12)$$

Where T is a threshold parameter that controls the sensitivity of the model to changes in the background.

Point classification Given the mixture of Gaussian model, we can check each pixel intensity X_t against the distributions:

- If X_t matches one of distributions, it is classified accordingly. X_t is said to match a distribution if the distance between X_t and the mean of the distribution is less than a certain threshold:

$$|X_t - \mu_i| < k\sigma_i$$

- If X_t does not match any of the distributions, the distribution with the lowest weight is replaced with a new distribution with mean equal to X_t , high standard deviation, and low weight.

Update Model parameters Also in this case, the parameters of the model are updated over time, to adapt to changes in the background and to changes in the illumination. In particular, we are interested in updating the weights of the distributions, since the means and the standard deviations are updated just as in the Gaussian average approach, but only for the distribution that matches the pixel intensity X_t .

The update of the weights can be formalized as:

$$w_{i,t} = (1 - \alpha)w_{i,t-1} + \alpha M \quad (5.13)$$

Where M is a binary variable that indicates whether the distribution matches the pixel intensity X_t (1 if it matches, 0 otherwise). This update allows to increase the weight of the distribution that matches the pixel intensity X_t , and to decrease the weight of the distributions that do not match the pixel intensity X_t .

6 Motion Tracking

While motion detection identifies the presence of movement, **motion tracking** aims to follow specific objects across consecutive frames. This process involves establishing temporal correspondence by associating detected entities in the current frame with their instances in subsequent frames.

Motion tracking is essential across various domains, including:

- **People monitoring:** analyzing crowd dynamics or tracking a specific individual;
- **Vehicle tracking:** monitoring the movement of vehicles in traffic surveillance;
- **Biological applications:** tracking the movement of animals or cells in scientific research;

The primary objective is often **object tracking**, which maintains the persistent identity of a target over time. Depending on the application, tracking can be performed in different dimensionalities:

- Tracking 2D coordinates in the image plane (e.g., using bounding boxes or centroids);
- Tracking 3D coordinates in real-world space (requires multiple cameras or depth sensors);
- Determining the pose or articulation of complex objects, such as human skeletal tracking.

6.1 2D Tracking Methodologies

In scenes with a single moving object, tracking can often be achieved by simply following the object's centroid. However, multi-object scenarios require sophisticated association algorithms to prevent identity switches. Common 2D tracking approaches include:

- **Region-based:** Associating areas based on global features like color or texture.
- **Contour-based:** Following the evolving boundary or "snake" of an object.
- **Feature-based:** Tracking distinct interest points, such as Harris corners or SIFT features.
- **Template-based:** Matching a predefined visual patch of the target across frames.

6.2 Region-Based Tracking

Region-based tracking follows image patches with a **uniform appearance**. This method is computationally efficient and well-suited for real-time systems, providing a robust balance between speed and discriminative power.

The standard implementation utilizes a **color histogram** as a visual signature. The workflow typically involves:

1. Segmenting the region of interest (ROI) from the background.
2. Computing the color distribution (histogram) of the ROI.
3. Searching for the most similar histogram in the subsequent frame to locate the object.

While effective for targets with distinctive colors, this approach is sensitive to illumination changes. To mitigate this, practitioners often employ the **HSV color space** (which separates luminance from chromaticity) or implement **dynamic model updates** to adapt the histogram over time.

6.2.1 Part-Based Models

A single global histogram may fail when tracking complex, non-rigid objects like humans. **Part-based models** decompose the object into several sub-regions (e.g., head, torso, and legs), each with its own local histogram.

This decomposition provides two major advantages:

- **Robustness to Occlusion:** If the legs are hidden by an obstacle, the system can maintain the track using the head and torso signatures.
- **Spatial Verification:** It prevents confusion between objects with similar colors but different configurations. Take in consideration the case of two people, one wearing a white shirt and black pants, and the other wearing a black shirt and white pants. A global histogram might struggle to differentiate them, but a part-based model can leverage the spatial arrangement of colors to maintain accurate tracking.

6.2.2 Implementation and Similarity Metrics

The tracking process evaluates the similarity between the **tracked model** (O^t) in the current frame and a stored **reference model** (O^r).

Common metrics for histogram comparison include:

- **Histogram Intersection:** Measures the correlation between two distributions.

$$\cap(O^t, O^r) = \sum_{n=1}^U \min(O_n^t, O_n^r)$$

This simply returns the area of the intersection of the two histograms, which is a measure of their similarity.

- **Sum of Squared Differences (SSD):** Measures the Euclidean distance between histogram bins.

$$SSD(O^t, O^r) = \sum_{n=1}^U (O_n^t - O_n^r)^2$$

This instead returns the area of the difference between the two histograms, which is a measure of their dissimilarity.

To improve robustness, bins typically represent color ranges rather than individual values to account for sensor noise and minor lighting fluctuations.

6.2.3 Addressing Environmental Noise: Shadows

Shadows represent a significant challenge in region-based tracking because they alter the pixel values of the background, often leading to false positives in motion detection.

To prevent a shadow from being tracked as a separate entity or distorting the object's histogram, we assume that a shadow primarily reduces **brightness** while leaving **chromaticity** (Hue and Saturation) relatively intact. By performing tracking in the HSV space and ignoring the Value (V) component, the system can become significantly more shadow-invariant, improving the robustness of the tracking process in outdoor environments or scenes with variable lighting conditions.

6.2.4 Blob Extraction and Post-Processing

Following motion detection, the resulting binary masks often contain "noise"—fragmented regions, isolated pixels, or holes within moving objects caused by imperfect segmentation. To transform these raw pixels into actionable data, we perform **blob extraction** (also known as Connected Component Labeling).

This process aggregates neighboring foreground pixels into coherent, labeled regions called **blobs**, which represent the physical objects in the scene. To ensure these blobs accurately reflect the geometry of the targets, the extraction process typically involves the following refinement steps:

- **Morphological Filtering:** Applying *erosion* to remove isolated noise (small pixel clusters) and *dilation* to bridge gaps and fill internal holes within an object.
- **Connectivity Analysis:** Grouping pixels based on 4-way or 8-way adjacency to assign a unique ID to each spatially distinct moving entity.
- **Feature Extraction:** Once a blob is defined, we calculate its geometric properties, such as the **centroid**, **bounding box**, **area**, and **aspect ratio**.

By filtering out blobs that are too small to be objects of interest, we provide the downstream tracking algorithms with a clean, high-level representation of the scene.

6.2.5 Target Association

Once we have the blobs representing the detected objects, we need to associate them across consecutive frames to maintain the identity of each object over time. This process is known as **target association** and is actually common to all tracking methods, not just region-based ones.

In general, detection is not carried out on a frame-by-frame basis, but rather on a **temporal window** of frames, since it can be demanding in terms of computational resources. This means that the tracking algorithm must be able to handle cases where an object is not detected in every single frame, which adds complexity to the association process.

Correspondence is typically achieved through **data association algorithms** that match the detected blobs based on their spatial proximity and appearance features. Such methods rely on the **optical flow** assumption, which states that the motion of objects between frames is small enough to allow for reliable matching based on their positions and appearances. It's important to keep in mind that cameras and frame rates must be chosen to satisfy this assumption, as large displacements can lead to tracking failures.

Different approaches to target association include:

- **Nearest Neighbor:** Assigning each detected blob to the closest track based on spatial distance between centroids;
- **Overlapping:** Matching blobs based on the degree of overlap between their bounding boxes. This has issues with scale changes;
- **Bounding Box - Overlap:** Similar to the previous method, but consider the bounding box of the track instead of the blob.

There are no best practices for target association, and the choice of method depends on the specific application and the characteristics of the scene being analyzed.

6.2.6 Splitting and Merging

Very similar but distinct problems that can arise during tracking are **splitting** and **merging**.

Splitting When multiple blobs are present in the same frame, we might encounter one of two problematic scenarios:

1. **Single object - multiple blobs:** this occurs when an object is fragmented into several blobs due to imperfect segmentation.
2. **Multiple objects - single blob:** this happens when two or more objects are merged into a single blob, often due to occlusion or proximity.

These 2 situations are different faces of the same problem, which is called **splitting**. Before understanding which of the two cases we are facing, evidence need to be collected over a temporal window of frames. This way, we can analyze the behavior of the blobs over time to determine whether they represent a single object or multiple objects.

Merging On the other hand, **merging** occurs when multiple objects are detected as a single blob, due to proximity or occlusion. An example of this is when a person which is entering a scene. Initially, feet and arms might be detected as separate blobs. As the person moves further into the scene, these blobs might merge into a single blob representing the whole body.

In both cases, its required to analyze the temporal evolution of the blobs to determine whether they represent a single object or multiple objects, and to adjust the tracking accordingly.

6.2.7 Occlusion

Moving object can be partially or fully occluded by each other when they are in close proximity. This can lead to tracking failures, as one object disappears from the scene, and bigger blobs may appear, which might have no resemblance to the original objects.

This is something that need to be handled before the occlusion occurs, by analyzing the proximity of the objects and predicting potential occlusions.

Usually, the system suspends the tracking of the involved objects during the occlusion, and then re-identifies them once they reappear as separate entities in the scene.

6.3 Feature Based Tracking

The idea behind feature-based tracking is to replace the information carried by the histogram of the whole object with a set of distinctive local **features**, extracted from the object itself. These features are typically points of interest, such as corners, edges, or blobs, that can be reliably detected and tracked across frames.

Considering a set of images $A = \{I_1, I_2, \dots, I_n\}$, we can extract a set of features F_i from each image I_i . The position of each feature in a given image can be represented as a vector of coordinates $f_i(x_i, y_i)$. The objective of feature-based tracking is to determine the displacement vector $d_i = (dx_i, dy_i)$ that best estimates the movement of feature f_i from one frame to the next, such that:

$$f_{i+1}(x_i + dx_i, y_i + dy_i)$$

6.3.1 Good Features to Track

Not all features are equally suitable for tracking. The **Good Features to Track** algorithm identifies features that are likely to be stable and distinctive across frames, making them ideal candidates for tracking. This method is based on the concept of **corner detection**, which identifies points in the image where the intensity changes significantly in multiple directions.

For each pixel in the image $p(x, y)$ we compute the **structure tensor** Z :

$$Z = \begin{bmatrix} \sum_W J_x^2 & \sum_W J_x J_y \\ \sum_W J_x J_y & \sum_W J_y^2 \end{bmatrix}$$

where J_x and J_y are the gradients of the image in the x and y directions, respectively, and the summation is performed over a window W centered around the pixel p .

The eigenvalues of the structure tensor, λ_1 and λ_2 , provide a measure of the local intensity variation around the pixel. A pixel is considered a good feature to track if the smallest eigenvalue $\lambda_{min} = \min(\lambda_1, \lambda_2)$ is above a certain threshold, indicating that changes are in multiple directions, which is characteristic of corners.

6.3.2 Tracking Features

Once we have identified the good features to track, we need to track their movement across frames. Ideally, we would expect that the position of a feature in the next frame can be predicted by adding a displacement vector to its current position:

$$I_i(Df - d) = I_{i+1}(f)$$

where I_i, I_{i+1} are successive frames, f is the position of the feature in the current frame, D is the deformation matrix and d is the displacement vector we want to estimate.

However, due to the noise, the equality does not hold perfectly. Assuming that the motion between frames is small, we can use a translational model to approximate the movement of the feature, which simplifies the problem to finding the displacement vector d that minimizes the difference between the two images:

$$\epsilon = \int \int_W [I_i(f - d) - I_{i+1}(f)]^2 \omega(f) df$$

The value of d that minimizes this error can be found by simply taking the derivative of ϵ with respect to d and setting it to zero.

6.4 Lucas-Kanade Optical Flow

The Lucas-Kanade method is an improvement over the basic feature tracking approach. It uses a two-frame differential method to estimate the optical flow, which is the apparent motion of brightness patterns in the image.

Consider $u = (u_x, u_y)$ in frame I_i and $v = (v_x, v_y)$ in frame I_{i+1} . The goal is to find the displacement vector d such that:

$$v = u + d$$

Where d is the vector that minimizes:

$$\epsilon(d) = \epsilon(d_x, d_y) = \sum_{x=u_x-\gamma_x}^{u_x+\gamma_x} \sum_{y=u_y-\gamma_y}^{u_y+\gamma_y} [I_i(x, y) - I_{i+1}(x + d_x, y + d_y)]^2$$

Where γ is the size of the window around the feature point that we are considering for tracking.

The strong point of the Lucas-Kanade method comes from its pyramidal implementation, which allows it to handle larger displacements by creating a multi-scale representation of the image.

6.4.1 Pyramidal implementation

The two key components of any feature tracker are **accuracy** and **robustness**.

- **Accuracy:** relates to the ability to recreate as perfectly as possible the motion of the feature. Usually, a smaller window size γ is preferable to increase accuracy, since it focuses on a smaller area around the feature point.

- **Robustness:** relates to the sensitivity of the tracking algorithm to big displacements. Usually, a larger window size γ is preferable to increase robustness.

We can notice that the two objectives are in conflict with each other, since increasing the window size to improve robustness will decrease accuracy in detecting small movements, and vice versa.

The pyramidal implementation of the Lucas-Kanade method addresses this trade-off by creating a multiscale representation of the image, where each level of the pyramid corresponds to a different resolution.

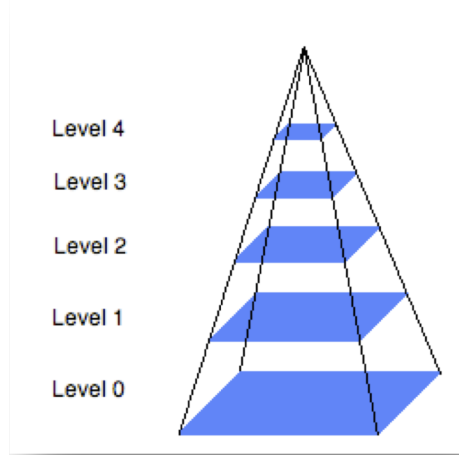


Figure 18: Pyramidal implementation of the Lucas-Kanade method.

We have that:

- At level 0, we have the original image with the highest resolution, which allows for accurate tracking of small movements;
- At higher levels, we have downsampled versions of the image. Due to the reduced resolution, bigger movements are now represented as smaller pixel displacements, allowing for more robust tracking of larger movements.

The L -th level of the pyramid is defined as a linear combination of the elements of the previous level, with more weight given to the central pixels and less weight given to the peripheral pixels.

$$I^L(x, y) = \frac{1}{4}I^{L-1}(2x, 2y) + \frac{1}{8}I^{L-1}(2x+1, 2y) + \frac{1}{8}I^{L-1}(2x, 2y+1) + \frac{1}{16}I^{L-1}(2x+1, 2y+1)$$

At each level of the pyramid, the Lucas-Kanade method is applied to estimate the displacement vector d for the features. The estimated displacement is then

refined by propagating it down the pyramid:

$$d^{L-1} = 2(d^L + \delta^L)$$

Where:

- d^L is the displacement estimated at level L . It is multiplied by 2 to account for the fact that the resolution at level L is half of the resolution at level $L - 1$, so the displacement needs to be scaled accordingly.
- δ^L is the correction term computed at level L to fine-tune the estimate.

6.5 Bayesian Tracking

Bayesian tracking is a probabilistic approach to motion tracking that models the uncertainty in the tracking process. It tries to estimate the state of the system over discrete time steps, given a sequence of observations.

State The state of the system at time t is represented by a random variable X_t , which includes information about the position, velocity, and acceleration along each axis of the tracked object. It can be represented as a function of the previous state and a process noise term:

$$X_t = f_t(X_{t-1}, w_{t-1})$$

Where:

- f_t is the non-linear state transition function that models how the state evolves over time;
- w_{t-1} is the process noise, representing the uncertainty in the state transition;
- X_{t-1} is the state of the system at the previous time step.

Take as example the case of a person walking in a scene. If no noise is involved, the state transition function f_t would simply predict the next position of the person based on their current velocity and direction of movement. Instead, we need to account for the fact that the person's movement might not be perfectly predictable, and that there might be external factors that can affect their trajectory (noise).

Observation The observations at time t are represented by another random variable Z_t , which includes the measurements obtained from the sensors. It can be represented as a function of the current state and an observation noise term:

$$Z_t = h_t(X_t, v_t)$$

Where:

- h_t is the non-linear observation function that models how the state is transformed into observations;
- v_t is the observation noise, generated by the sensors and the environment;
- X_t is the state of the system at the current time step.

6.5.1 How it works

The Bayesian tracking algorithm is a recursive filter that updates the system's state estimate whenever a new observation Z_t is recorded. It refines the predicted state X_t by correcting it with the new measurement.

The process begins with an initial probability distribution, or **prior**, $p(X_0)$, representing the baseline knowledge of the system.

The objective is to compute the **posterior distribution** $p(X_t|Z_t)$, which represents the updated belief of the state at time t given all the measurements up to that point.

The estimation cycle consists of two main steps:

1. **Prediction:** predict the next state based on the previous state and the state transition model, resulting in a prior distribution (not based on the observation) for the current time step.
2. **Update:** refine the predicted state using the new observation and the observation model, resulting in the posterior distribution (based on the observation) for the current time step.

Prediction Step Using the state transition function from 6.5, the evolution of the system can be described by the conditional probability distribution:

$$p(X_t|X_{t-1})$$

which models the likelihood of transitioning from state X_{t-1} to state X_t .

Using the observation function from 6.5, the likelihood of observing the measurement Z_t , given the current state X_t can be expressed as:

$$p(Z_t|X_t)$$

To predict the state at time t , the algorithm propagates the posterior probability, obtained from the previous time step $p(X_{t-1}|Z_{1:t-1})$ forward. This results in the **prior distribution** for the current time step, which is computed by marginalizing over all possible previous states:

$$p(X_t|Z_{t-1}) = \int p(X_t|X_{t-1})p(X_{t-1}|Z_{t-1})dX_{t-1}$$

Update Step When the new observation Z_t becomes available, the likelihood $p(Z_t|X_t)$ is derived from the observation model in 6.5.

The prediction is then updated using Bayes' rule to obtain the new posterior:

$$p(X_t|Z_t) = \frac{p(Z_t|X_t)p(X_t|Z_{t-1})}{p(Z_t|Z_{t-1})}$$

where the denominator $p(Z_t|Z_{t-1})$ acts as a normalizing constant.

6.6 Kalman Filter

The Kalman Filter is a specific implementation of the Bayesian tracking framework, designed for linear systems with **Gaussian noise**.

In line to what we have seen in the bayesian tracking section, the Kalman Filter still consists of:

- A prediction of the next state (prior distribution);
- An update of the state estimate based on new observations (posterior distribution).

It heavily relies on the assumption that both the process noise and the observation noise are Gaussian, which allows for a closed-form solution to the estimation problem.

The algorithm starts from a first measurement Z_0 , with its associated noise variance $\sigma_{z_0}^2$. It assumes that the first prediction is equal to the first measurement, and that the first prediction error is equal to the noise variance of the first measurement:

$$\begin{aligned}\hat{X}_0 &= Z_0 \\ \sigma_0 &= \sigma_{z_0}^2\end{aligned}$$

The Kalman Filter then iteratively performs the prediction and update steps as new measurements become available, propagating the state estimate and its associated uncertainty over time. Given a new measurement Z_t $\sigma_{z_t}^2$ at time t , the Kalman Filter performs the following steps:

1. **Prediction:** we add a displacement to the previous state estimate

$$\hat{X}_t = \hat{X}_{t-1} + K_t(Z_t - \hat{X}_{t-1})$$

where K_t is the **Kalman Gain**, which weights the influence of the new measurement relative to the current state estimate. It is computed as:

$$K_t = \frac{\sigma_{z_{t-1}}^2}{\sigma_{z_{t-1}}^2 + \sigma_{z_t}^2}$$

We can see the prediction as a weighted average between the previous state estimate and the new measurement, where the weights are determined by the relative uncertainties of the two sources of information.

2. **Update:** we update the error variance of the state estimate:

$$\frac{1}{\sigma_t^2} = \frac{1}{\sigma_{z_{t-1}}^2} + \frac{1}{\sigma_{z_t}^2}$$

The Kalman Filter provides a computationally efficient solution to the least-squares estimation problem. In particular, we are trying to find the Kalman Gain K_t that minimizes the mean squared error of the state estimate $\hat{X}_t$.

6.6.1 Kalman Filter in Practice

Now that we have seen the mathematical formulation of the Kalman Filter, let's discuss how it can be applied in practice for motion tracking.

State Model The state model can be represented as a linear function of the previous state and the control input, plus some process noise:

$$x_t = A_t x_{t-1} + B_t u_t + w_{t-1}$$

Where:

- x_t is the state vector at time t ;
- A_t is the state transition matrix that models how the state evolves from time $t - 1$ to time t ;
- B_t is the control input matrix that models how external inputs u_t affect the state. For example, in the case of a vehicle tracking scenario, u_t could represent the acceleration or steering commands that influence the vehicle's motion;
- w_{t-1} is the process noise, assumed to be Gaussian with zero mean and covariance Q_t ;

Usually, if we have no control input, we can simply ignore the term $B_t u_t$ in the state model, which simplifies it to:

$$x_t = A_t x_{t-1} + w_{t-1}$$

Observation Model The observation model can be represented as a linear function of the state, plus some observation noise:

$$z_t = H_t x_t + v_t$$

Where:

- z_t is the observation vector at time t ;
- H_t is the observation matrix that models how the state is transformed into observations;
- v_t is the observation noise, assumed to be Gaussian with zero mean and covariance R_t ;

6.6.2 The Recursive Stages

The filter iterates through the following equations to propagate the state estimate $\hat{x}_t$ and the error covariance matrix P_t :

1. Prediction Project the state and uncertainty forward, computing the prior state estimate $\hat{x}_t^-$ and its associated error covariance P_t^- :

$$\begin{aligned}\hat{x}_t^- &= A_t \hat{x}_{t-1} + B_t u_t \\ P_t^- &= A_t P_{t-1} A_t^T + Q_t\end{aligned}$$

2. Update Once the new observation z_t is available, we compute the Kalman Gain K_t , update the state estimate $\hat{x}_t$, and refine the error covariance P_t :

- **Kalman Gain:** $K_t = P_t^- H_t^T (H_t P_t^- H_t^T + R_t)^{-1}$
- **State Update:** $\hat{x}_t = \hat{x}_t^- + K_t (z_t - H_t \hat{x}_t^-)$
- **Covariance Update:** $P_t = (I - K_t H_t) P_t^-$

6.6.3 Extended Kalman Filter

In the case of non-linear systems, the standard Kalman Filter is not applicable, since it relies on linearity assumptions. In object tracking scenarios, we often encounter non-linear dynamics and measurement models, which necessitate the use of the **Extended Kalman Filter** (EKF).

The EKF linearizes the non-linear functions around the current state estimate, allowing us to apply the Kalman Filter framework to non-linear problems. This is done by computing the **Jacobian matrices** of the state transition and observation functions, which serve as linear approximations of the non-linear models.

6.7 Particle Filter

Even though the EKF can handle non-linear systems, it still relies on the assumption of the optical flow and of Gaussian noise, which may not always hold in real-world tracking scenarios.

The **Particle Filter** is a more general approach that can handle non-linear and non-Gaussian systems. This is done by representing the set of possible alternative states of the system as a collection of samples, called **particles**, rather than a single state estimate with an associated covariance. Each particle represents a possible state of the system, and is associated with a weight that reflects the likelihood of that state given the observations.

The higher the number of particles, the closer our approximation will be to the true posterior distribution, but this also increases the computational cost of the algorithm.

6.7.1 Theoretical Definition

The Particle Filter defines a set of N particles $\{x_t^{(i)}, w_t^{(i)}\}_{i=1}^N$, where $x_t^{(i)}$ is the state of the i -th particle at time t , and $w_t^{(i)}$ is its associated weight.

These particles are selected from a proposal distribution $\pi(x_t|x_{t-1})$, which is chosen as an hyperparameter of the algorithm.

The a posteriori distribution of the state given the observations can be approximated as:

$$p(x_t|z_t) \approx \sum_{i=1}^N w_{t-1}^{(i)} \delta(x_t - x_t^{(i)})$$

where $\delta(x_t - x_t^{(i)})$ represents the distance between the state x_t and the state of the i -th particle $x_t^{(i)}$.

To update the particles and their weights, we perform the following steps:

1. **Sampling:** For each particle, we sample a new state from the proposal distribution;
2. **Weighting:** We update the weight of each particle based on the likelihood of the new state given the observation:

$$w_t^{(i)} = w_{t-1}^{(i)} p(z_t|x_t^{(i)})$$

3. **Normalization:** the weights are normalized to ensure that they sum to 1. This is done by dividing each weight by the sum of all weights:

$$w_t^{(i)} = \frac{w_t^{(i)}}{\sum_{j=1}^N w_t^{(j)}}$$

4. **Resampling:** To prevent the problem of particle degeneracy, where a few particles have significant weights while others have negligible weights, we perform resampling. This involves drawing a new set of particles from the current set, with replacement, according to their weights. Each new particle is assigned a weight of $\frac{1}{N}$, and the process is repeated for the next time step.

This way, at each iteration, more particles are concentrated in regions of the state space that are more likely given the observations, allowing for a more accurate approximation of the posterior distribution.

For example, if we are tracking a person in a scene, and we have a new observation that indicates the person's direction of movement, the particles that are aligned with that direction will receive higher weights, while particles that are not aligned will receive lower weights and are less likely to be resampled in the next iteration.

7 Camera Geometry

We have already seen that the camera is a projection device. The location of the pixel in the image plane has a direct relationship with the location of the point in the 3D world. When we approximate the camera with a pinhole model, we might lose some information about the scene, such as the depth of the objects.

The position of the acquisition device plays a crucial role in the way we perceive the world. In this section we will try to give some explanation of these phenomena and try to give an explanation of how the camera geometry works.

7.1 2D Case

Typically, a pixel is represented as a 2D point in the image plane using its coordinates.

$$p = [x, y]^T = \begin{bmatrix} x \\ y \end{bmatrix}$$

Often, we use homogeneous coordinates to represent points in the image plane. This allows us to represent points at infinity and perform projective transformations more easily. In homogeneous coordinates, a point is represented as a three-dimensional vector:

$$p = [sx, sy, s]^T$$

where s is a scaling factor, commonly set to 1.

7.1.1 Affine Transformation

An affine transformation is a spatial transformation, represented as a multiplication of the **homogeneous point** with some predefined matrix. We can define different types of transformations, such as **scaling**, **rotation**, **translation** and a **composition** of them.

Scaling Scaling is a transformation that changes the size of an object, by multiplying the coordinates of each point by a scaling factor c . It is applied to all points in the image either uniformly or non-uniformly. The scaling transformation matrix S_{c_x, c_y} is defined as:

$$S_{c_x, c_y} = \begin{bmatrix} c_x & 0 \\ 0 & c_y \end{bmatrix}$$

where c_x and c_y are the scaling factors for the x and y directions, respectively. If the scaling factors are the same, the transformation is uniform.

The new coordinates (x', y') of the point after scaling can be calculated as:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} c_x & 0 \\ 0 & c_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

We can express the new coordinates (x', y') of the point after scaling as:

$$x' = c_x \cdot x$$

$$y' = c_y \cdot y$$

Rotation Rotation is a transformation that changes the orientation of an object by rotating it around a specific point, usually the origin.

The rotation transformation matrix R_θ for a rotation by an angle θ is defined as:

$$R_\theta = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

where θ is the angle of rotation in radians.

The rotation transformation can be applied to a point (x, y) in the image plane to obtain the new coordinates (x', y') after rotation:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

The new coordinates (x', y') of the point after rotation can be calculated as:

$$x' = x \cdot \cos \theta - y \cdot \sin \theta$$

$$y' = x \cdot \sin \theta + y \cdot \cos \theta$$

Translation Translation is a transformation that moves an object from one location to another without changing its shape or orientation. It is represented as a **shift** in the coordinates of the points in the image, equivalent to changing the origin of the coordinate system.

We basically want to move the point (x, y) to a new location (x', y') by adding a translation vector (x_0, y_0) :

$$x' = x + x_0$$

$$y' = y + y_0$$

To do this, we can represent the translation transformation using homogeneous coordinates, which allows us to express it as a matrix multiplication. The translation transformation matrix D_{x_0, y_0} can be represented as:

$$D_{x_0, y_0} = \begin{bmatrix} 1 & 0 & x_0 \\ 0 & 1 & y_0 \\ 0 & 0 & 1 \end{bmatrix}$$

The new coordinates (x', y') of the point after translation can be calculated as:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & x_0 \\ 0 & 1 & y_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

where x_0 and y_0 are the translation distances in the x and y directions, respectively.

7.1.2 Camera Matrix

All the transformations we have seen so far can be combined into a single transformation matrix, called the **camera matrix**. This allows us to represent the relationship between the 3D world and the 2D image plane in a compact form, allowing to correctly map points from the 3D world to the 2D image plane.

The camera matrix is typically a combination of 4 parameters: rotation angle, translation vector, and the scaling factor.

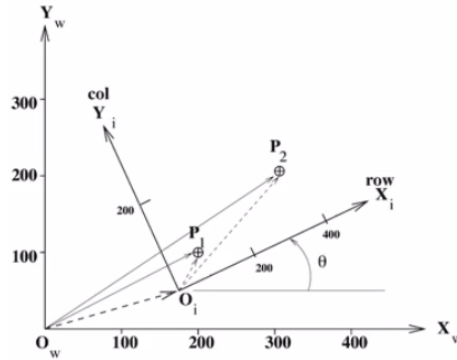


Figure 19: Camera matrix in the 2D case.

From the example, we can see how the point P_1 , from the real world, is projected onto the image plane X_i, Y_i as the point P_2 . This is done by applying a translation from the real world coordinates to the camera coordinates, then applying a rotation of angle θ and finally applying a scaling factor s to get the final coordinates of the point in the image plane.

Given a point P_j in the real world, we can express the transformation to the image plane as follows:

$$P_j = D_{x_0, y_0} R_\theta S_{s_x, s_y} P_i$$

where P_i is the point in the image plane, D_{x_0, y_0} is the translation matrix, R_θ is the rotation matrix, and S_{s_x, s_y} is the scaling matrix.

Compute the Camera Matrix We can compute the parameters of the camera matrix by using a set of known points, called **control points** in the real world and their corresponding points in the image plane.

Since we have 4 parameters to estimate, we need at least 4 equations to solve for these parameters. Each control point provides us with 2 equations (one for the x-coordinate and one for the y-coordinate), so we need at least 2 control points.

For each control point, we can write the following equations:

$$\begin{bmatrix} x_w \\ y_w \end{bmatrix} = \begin{bmatrix} 1 & 0 & x_0 \\ 0 & 1 & y_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$$

where (x_w, y_w) are the coordinates of the point in the real world, (x_i, y_i) are the coordinates of the point in the image plane, (x_0, y_0) are the translation parameters, (c_x, c_y) are the scaling parameters, and θ is the rotation angle.

7.1.3 General Camera Matrix

Given that the three matrices that we are multiplying $(R_\theta, S_{s_x, s_y}, D_{x_0, y_0})$ are all of the same size, we can combine them into a **single camera matrix** C :

$$C = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix}$$

Given the camera matrix C , we can compute the transformation from the real world to the image plane and vice versa. For example, given a point $P_j = \begin{bmatrix} x_j \\ y_j \\ 1 \end{bmatrix}$ in the real world, we can compute its projection on the image plane as follows:

$$P_i = CP_j$$

Least Square Approach To obtain the 6 parameters of the camera matrix C , we need at least 3 control points. A good rule of thumb is to select them in a way that they are not collinear or too close to each other, to avoid numerical instability in the estimation of the parameters. But, a more effective way to reduce the error in the estimation of the camera matrix is to use more than 3 control points and solve for the parameters using a least squares approach.

Our goal is to find the set of parameters that minimizes the **error** between the projected points and the actual points in the image plane. The error can be

defined as the sum of the squared distances between the projected points and the actual points in the image plane:

$$E = \sum_{j=1}^N (a_{11}x_j + a_{12}y_j + a_{13} - u_j)^2 + (a_{21}x_j + a_{22}y_j + a_{23} - v_j)^2$$

where (u_j, v_j) are the coordinates of the point in the image plane corresponding to the control point $P_j = [x_j \ y_j \ 1]$ in the real world.

The solution to this optimization problem can be found by computing the partial derivatives of the error function with respect to the parameters of the camera matrix, setting them to zero, and solving the resulting system of equations.

The resulting equation system is:

$$\begin{bmatrix} \sum x_j^2 & \sum x_j y_j & \sum x_j & 0 & 0 & 0 \\ \sum x_j y_j & \sum y_j^2 & \sum y_j & 0 & 0 & 0 \\ \sum x_j & \sum y_j & N & 0 & 0 & 0 \\ 0 & 0 & 0 & \sum x_j^2 & \sum x_j y_j & \sum x_j \\ 0 & 0 & 0 & \sum x_j y_j & \sum y_j^2 & \sum y_j \\ 0 & 0 & 0 & \sum x_j & \sum y_j & N \end{bmatrix} \begin{bmatrix} a_{11} \\ a_{12} \\ a_{13} \\ a_{21} \\ a_{22} \\ a_{23} \end{bmatrix} = \begin{bmatrix} \sum x_j u_j \\ \sum y_j u_j \\ \sum u_j \\ \sum x_j v_j \\ \sum y_j v_j \\ \sum v_j \end{bmatrix}$$

This system of equations can be solved using standard linear algebra techniques, such as Gaussian elimination or singular value decomposition, to find the parameters of the camera matrix C .

7.2 3D Case

What we have seen so far is a simplified version of the camera geometry, where we have only considered the 2D case. This was only applicable to the case where the camera is looking at a flat surface, which is perfectly parallel to the image plane.

In the real world, we usually have a 3D scene and the camera is looking at it from a certain point of view. In this case, we also need to consider the **calibration** of the camera, which is the process of estimating the **intrinsic** parameters of the camera, and the **extrinsic** parameters.

7.2.1 Calibration

As we just said, we need to consider both intrinsic and extrinsic parameters of the camera to correctly model the camera geometry in the 3D case.

Intrinsic set of parameters that describe the internal characteristics of the camera, and are specific to the sensor itself. They include parameters such as the focal length, image distortion, and the size of the pixels.

These parameters are crucial for accurately modeling the 3D geometry of the scene. In fact, since each camera has its own intrinsic parameters, they will affect the outcome of the projection of the 3D points onto the image plane.

Extrinsic set of parameters that describe the position and orientation of the camera in the 3D world. They include parameters such as the rotation and translation of the camera with respect to a reference coordinate system.

Calibration is the process of estimating both intrinsic and extrinsic parameters of the camera, that allow us to correctly map the 3D points in the world to the 2D points in the image plane.

7.2.2 3D representation

In the 3D case, we can represent a point in the world as a 3D vector:

$$P = [X, Y, Z]^T$$

where X, Y, Z are the coordinates of the point in the 3D space.

We can also represent the point in homogeneous coordinates as:

$$P = [sX, sY, sZ, s]^T$$

where s is a scaling factor, commonly set to 1.

In general, each point has 4 coordinates systems:

- **Model coordinate system:** the coordinate system defined by the 3D model of the scene, with the origin at a specific point in the scene and the axes aligned with the model;
- **World coordinate system:** the real-world coordinate system;
- **Camera 1 coordinate system:** the coordinate system defined by one camera;
- **Camera 2 coordinate system:** the coordinate system defined by another camera;

Since the two cameras can have opposite views of the scene, there is no guarantee that there exists a one-to-one mapping for all the points in the scene. In fact, some points might be visible in one camera and not in the other, due to occlusions or the limited field of view of the cameras. For this kind of points, we cannot find their corresponding points in the 3D space.

7.2.3 3D affine transformation

Similar to the 2D case, we can also define affine transformations in the 3D case. The main difference is that we need to consider the z-coordinate of the points in the 3D space, and the transformation matrices will be of size 4x4 instead of 3x3.

Note

In the following section we will use a slightly different notation from the previous one. In particular, we represent as:

$${}^k P_i$$

the pixel P_i in the coordinate system of the camera k .

Translation The translation transformation, mapping a point ${}^1 P_i$ in the coordinate system of camera 1 to a point ${}^2 P_i$ in the coordinate system of camera 2, can be represented as:

$${}^2 P_i = T(t_x, t_y, t_z) {}^1 P_i$$

where $T(t_x, t_y, t_z)$ is the translation matrix defined as:

$$T(t_x, t_y, t_z) = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where (t_x, t_y, t_z) are the translation distances in the x, y, and z directions, respectively.

Scaling The scaling transformation, mapping a point ${}^1 P_i$ in the coordinate system of camera 1 to a point ${}^2 P_i$ in the coordinate system of camera 2, can be represented as:

$${}^2 P_i = S(s_x, s_y, s_z) {}^1 P_i$$

where $S(s_x, s_y, s_z)$ is the scaling matrix defined as:

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where (s_x, s_y, s_z) are the scaling factors in the x, y, and z directions, respectively.

Rotation The rotation transformation needs to consider the three angles of rotation around the x, y, and z axes.

The rotation transformation, mapping a point 1P_i in the coordinate system of camera 1 to a point 2P_i in the coordinate system of camera 2, can be represented as:

- Rotation around the x-axis:

$${}^2P_i = R(X, \theta_x) {}^1P_i$$

where $R(X, \theta_x)$ is the rotation matrix defined as:

$$R(X, \theta_x) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta_x & -\sin \theta_x & 0 \\ 0 & \sin \theta_x & \cos \theta_x & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where θ_x is the angle of rotation around the x-axis in radians.

- Rotation around the y-axis:

$${}^2P_i = R(Y, \theta_y) {}^1P_i$$

where $R(Y, \theta_y)$ is the rotation matrix defined as:

$$R(Y, \theta_y) = \begin{bmatrix} \cos \theta_y & 0 & \sin \theta_y & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta_y & 0 & \cos \theta_y & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where θ_y is the angle of rotation around the y-axis in radians.

- Rotation around the z-axis:

$${}^2P_i = R(Z, \theta_z) {}^1P_i$$

where $R(Z, \theta_z)$ is the rotation matrix defined as:

$$R(Z, \theta_z) = \begin{bmatrix} \cos \theta_z & -\sin \theta_z & 0 & 0 \\ \sin \theta_z & \cos \theta_z & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where θ_z is the angle of rotation around the z-axis in radians.

7.2.4 General Transformation Matrix

Similar to the 2D case, we can also define a general camera matrix in the 3D case, which combines all the transformations we have seen so far into a single matrix. The general camera matrix C can be represented as:

$$C = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This matrix allows us to represent the transformation from real from camera 1 points 1P_i to camera 2 points 2P_i as follows:

$${}^2P_i = C {}^1P_i$$

7.2.5 Camera Model

The General Transformation Matrix C is a powerful tool to represent the transformation between different camera coordinate systems, but it doesn't allow to map points from the 3D world to the 2D image plane, since it doesn't consider the intrinsic parameters of the camera.

To do this, we need to define a camera model that combines both the intrinsic and extrinsic parameters of the camera. Given the matrix ${}^I_W C$, we can map points from the world coordinate system ${}^W P$ to the image plane ${}^I P$ as follows:

$${}^I P = {}^I_W C {}^W P$$

where ${}^I_W C$ is the camera matrix that combines both intrinsic and extrinsic parameters of the camera, and ${}^W P$ is the point in the world coordinate system.

Formally, this transformation can be represented as:

$$\begin{bmatrix} s' P_x \\ s' P_y \\ s' \end{bmatrix} = {}^I_W C \begin{bmatrix} {}^W P_x \\ {}^W P_y \\ {}^W P_z \\ 1 \end{bmatrix}$$

7.2.6 Extension of the Camera Model

The previously defined camera model is a simplified version of the real camera model, which is based on the pinhole camera model. In fact, the points in the real world are considered as continuous, while the points in the image plane are discrete, since they are represented as pixels.

We need to apply some additional transformations to the camera model, such as:

- **Roto-Translation:** to account for the rotation and translation of the camera in the 3D world, we apply a roto-translation transformation to the camera model, which allows us to correctly map points from the world coordinate system $^W P$ to the camera coordinate system $^C P$.
- **Projection:** to map the 3D points in the camera coordinate system $^C P$ to the 2D points into an intermediate image plane $^F P$, we apply a projection transformation.
- **Pixel Scaling:** to account for the size of the pixels in the image plane, we apply a scaling transformation to the camera model, which allows us to correctly map points from the camera coordinate system $^F P$ (still in mm) to the image plane $^I P$ (in pixels).

This entire process gives us our final camera model, which allows us to correctly map points from the 3D world to the 2D image plane.

It can be defined as follows:

$$^I P = C^W P$$

where C is the final camera matrix that combines all the transformations we have seen so far, including the roto-translation, projection, and pixel scaling transformations.

$$C = \begin{bmatrix} c_{11} & c_{12} & c_{13} & c_{14} \\ c_{21} & c_{22} & c_{23} & c_{24} \\ c_{31} & c_{32} & c_{33} & 1 \end{bmatrix}$$